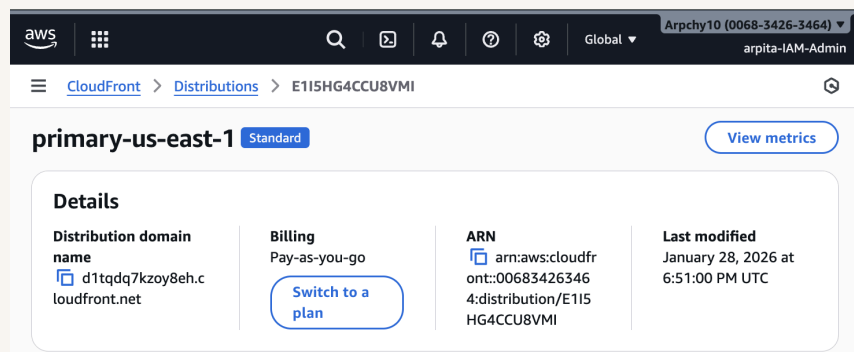




Build Instant Failover with CloudFront Origin Groups

CH

Arpita Chowdhury





Introducing Today's Project!

In this step, I'm going to create a CloudFront distribution by first gathering my two App Runner service URLs (my primary in us-east-1 and my secondary in us-west-2) and then configuring CloudFront with the primary origin so CloudFront can serve my app through a single global endpoint. CloudFront is important because it acts as a global routing + edge layer it can deliver responses from nearby edge locations for better performance and (in the next steps) enable automatic failover to the backup region without DNS delays.

Key services and concepts

The key tools I used include AWS CloudFront, App Runner, CloudWatch, and SNS to build a global failover architecture with monitoring and alerting. Key concepts I learned include CloudFront origin groups, automatic failover criteria based on HTTP error codes, edge-based routing, multi-region high availability, DNS-independent failover, latency testing across regions, and real-time alerting with CloudWatch alarms and SNS notifications. This project helped me understand how production systems achieve high availability and low latency at a global scale.

Challenges and wins

This project took me approximately 2 hours to complete. The most challenging part was configuring CloudFront origin groups and CloudWatch alarms correctly, especially understanding how failover criteria and metrics trigger automatic routing.



Arpita Chowdhury

NextWork Student

nextwork.org

It was most rewarding to see CloudFront instantly fail over to the secondary region when the primary service was paused, and then automatically fail back when it recovered, demonstrating real-world high availability and disaster recovery in action.



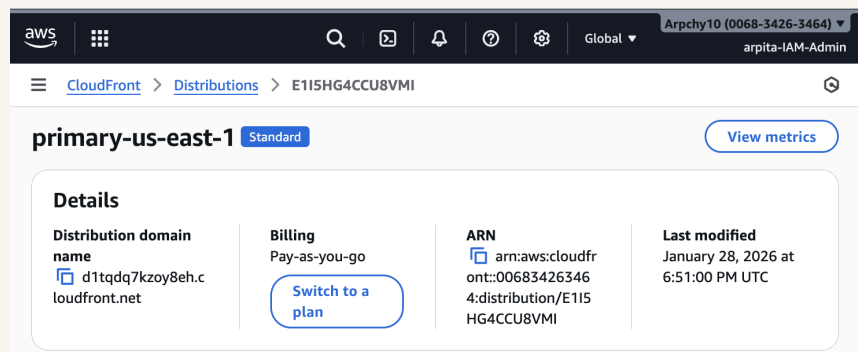
Setting Up Multi-Region Prerequisites

This project requires two running AWS App Runner services one deployed in us-east-1 and one in us-west-2 and each service must respond with its region name (e.g., "Hello from us-east-1!" / "Hello from us-west-2!"). I already have my App Runner services running in us-east-1 and us-west-2. These services are needed because CloudFront origin groups use them as the primary and backup origins, so CloudFront can health-check the primary and fail over to the secondary in seconds (and fail back automatically) without waiting for DNS propagation.



Creating a CloudFront Distribution

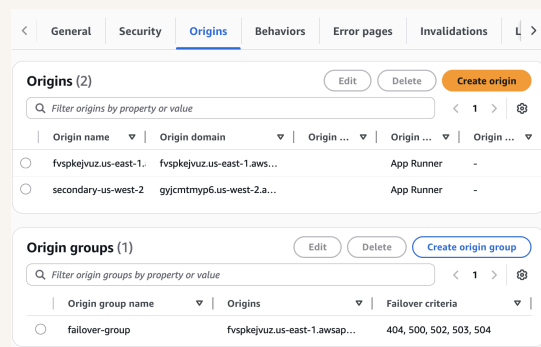
I created a CloudFront distribution named `primary-us-east-1` with my `us-east-1` App Runner service as the origin (`fvspkejvuz.us-east-1.awsapprunner.com`). My CloudFront URL (distribution domain name) is `d1tqdq7kzoy8eh.cloudfront.net`, which I can use to access the app through CloudFront's global edge network.





Configuring Origin Groups for Failover

In this step, I'm going to add a secondary origin (us-west-2 App Runner URL) and create an origin group that includes both the primary and secondary regions. Origin groups enable failover by allowing CloudFront to continuously health-check the primary origin and automatically route traffic to the secondary origin if the primary becomes unhealthy, without any DNS changes or manual intervention. Then, I'll configure the cache behavior to use this origin group so all requests go through CloudFront's failover logic and test the distribution to confirm traffic switches correctly when one region goes down.



Failover criteria configuration

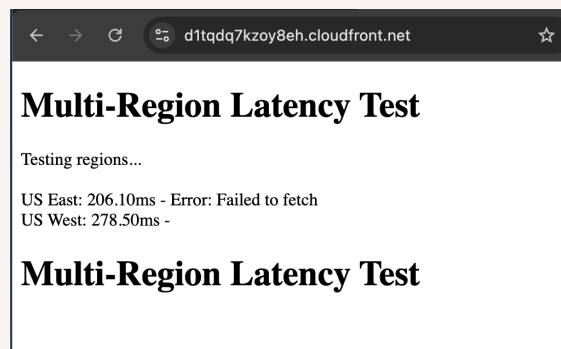
I created an origin group named failover-group with my primary origin in us-east-1 and my secondary origin in us-west-2. The failover criteria include HTTP 404, 500, 502, 503, and 504 errors because these status codes indicate the primary origin is unavailable or unhealthy, so CloudFront should automatically route traffic to the secondary region to maintain availability.

Testing the CloudFront URL



Testing Automatic Failover

In this step, I'm going to test failover by pausing the us-east-1 App Runner service to simulate a real outage and then refreshing my CloudFront URL to verify that traffic automatically switches to the us-west-2 secondary origin. After that, I'll verify failback by resuming the us-east-1 service and confirming that CloudFront automatically routes traffic back to the primary region. This tests CloudFront's origin group health checks, automatic failover, and failback behavior, demonstrating how edge-based routing can maintain high availability without DNS changes or manual intervention.

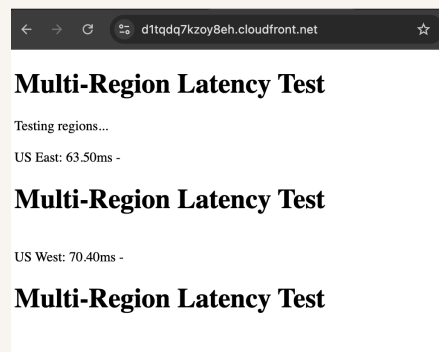


Simulating primary origin failure

I paused the us-east-1 App Runner service and CloudFront automatically detected the outage and failed over to us-west-2. The response changed from serving content from us-east-1 to showing an error for US East and successful responses from US West, confirming that CloudFront routed traffic to the secondary region when the primary became unavailable.



Verifying Automatic Failback



Restoring the primary origin

I resumed the us-east-1 App Runner service, and CloudFront automatically failed back to the primary region. The response returned to showing successful results from US East (with lower latency again) instead of only routing traffic to US West. This confirms that CloudFront origin groups support automatic failback, meaning traffic switches back to the primary origin once it becomes healthy again without any manual intervention.



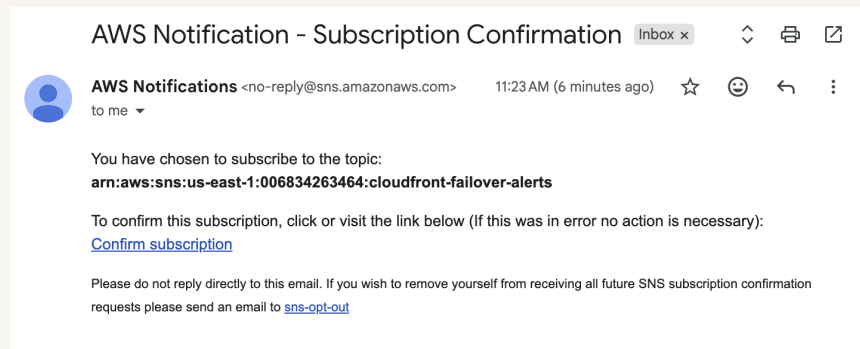
Building Production Alerting with CloudWatch

In this secret mission, I'm going to build alerting with CloudWatch and SNS to monitor CloudFront errors and notify me when something goes wrong. I will create a CloudWatch alarm on the 4xxErrorRate metric and configure SNS email notifications so I get an alert when error rates cross a defined threshold. This is important because automatic failover alone is not enough even if traffic switches to a healthy region, engineers still need real-time visibility and alerts to investigate outages, fix root causes, and maintain reliability and uptime SLAs in production systems.

Details			
Name CloudFront-Origin-Failover-Alert	State ⓘ Insufficient data	Namespace AWS/CloudFront	Datapoints to alarm 1 out of 1
Type Metric alarm	Threshold 4xxErrorRate > 50 for 1 datapoints within 1 minute	Metric name 4xxErrorRate	Missing data treatment Treat missing data as missing
Description Alerts when CloudFront origin error rate exceeds 50%, indicating a failover event	Actions ✅ Actions enabled	Region Global	Percentiles with low samples evaluate
	Last state update 2026-01-29 17:24:19 (UTC)	DistributionId E115HG4CCUBVMI	ARN arn:aws:cloudwatch:us-east-1:006834263464:alarm:CloudFront-Origin-Failover-Alert
		Statistic Average	
		Period 1 minute	

Configuring CloudWatch alarms

I created a CloudWatch alarm using the CloudFront 4xxErrorRate metric with a threshold that triggers when error rates exceed the defined limit within a one-minute period. The alarm is connected to an SNS topic to send email notifications whenever CloudFront detects elevated error rates, indicating a potential outage or failover event. This ensures I receive real-time alerts when the primary region becomes unhealthy and CloudFront automatically switches traffic to the secondary region.



Verifying alerting works

I tested the alarm by pausing the primary us-east-1 App Runner service to trigger CloudFront failover. After waiting for the metric evaluation period, I received an SNS email notification indicating elevated CloudFront error rates. This confirms that the CloudWatch alarm and SNS integration are working correctly and that I will be alerted in real time when a region becomes unhealthy and traffic fails over to the secondary region.



Wrapping Up

I did this project today to learn how to design a highly available multi-region architecture using CloudFront origin groups and automatic failover, so applications can stay online even when one region goes down. Another disaster recovery skill I want to learn is implementing active-active multi-region architectures with Route 53 health checks and database replication, so systems can handle regional failures with zero downtime and no manual intervention.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

